

Нейронные сети: от теории к практике

Материалы курса CS231n: Neural Networks



План презентации

1. Введение: что такое нейронная сеть?
2. Модель одного нейрона
3. Функции активации
4. Архитектура нейронных сетей
5. Прямое распространение (Forward Pass)
6. Предобработка данных
7. Инициализация весов
8. Регуляризация
9. Оптимизация и обучение
10. Практические рекомендации

Линейный классификатор:

$$s = Wx$$

- x — входной вектор (например, $[3072 \times 1]$ для CIFAR-10)
- W — матрица весов $[10 \times 3072]$
- s — вектор оценок классов $[10 \times 1]$

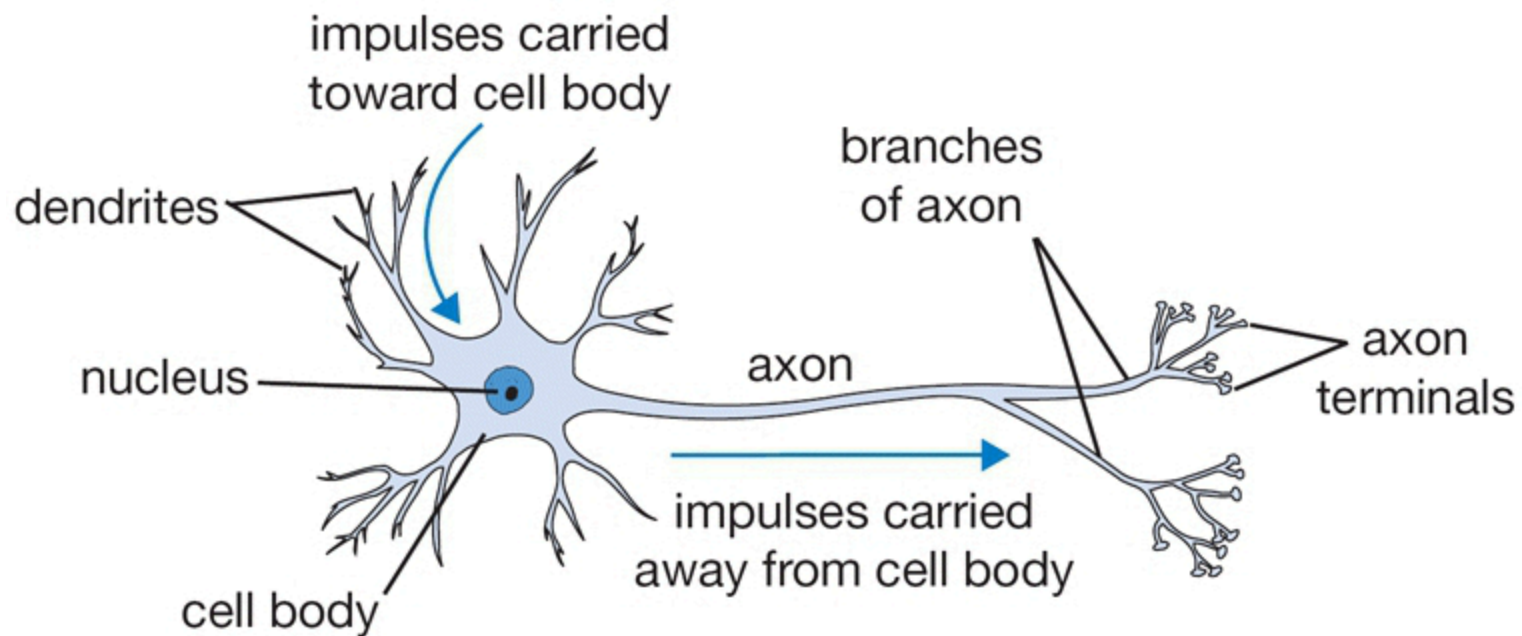
Нейронная сеть (2 слоя):

$$s = W_2 \cdot \text{softmax}(W_1 x)$$

- W_1 : $[100 \times 3072] \rightarrow$ промежуточный вектор $[100 \times 1]$
- $\text{softmax}(\theta, \cdot)$ — нелинейность
- W_2 : $[10 \times 100] \rightarrow$ выходные оценки $[10 \times 1]$

Ключевая идея: нелинейность позволяет моделировать сложные зависимости!

Модель одного нейрона



Биологический
нейрон

- Дендриты →
- Тело клетки
- Аксон →

Модель одного нейрона

Биологическая мотивация:

Математическая модель:

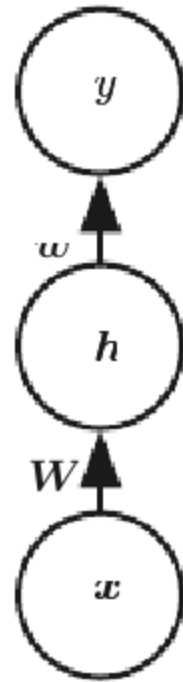
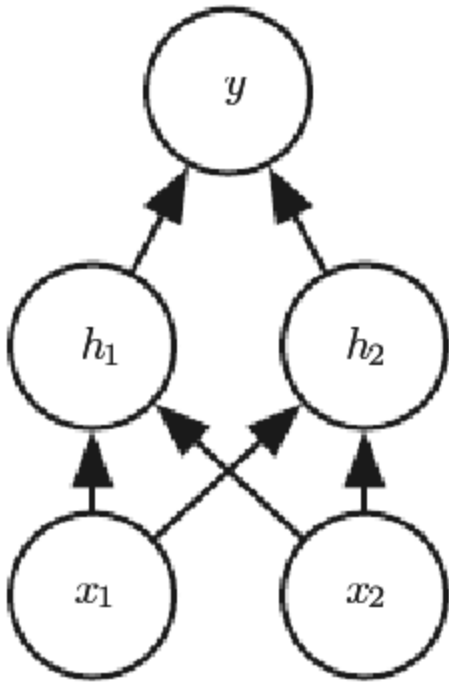
$$z = \sum_i (w_i \cdot x_i) + b$$

$$a = f(z) \leftarrow \text{функция активации}$$

Вычисление нейрона (код):

```
class Neuron:
    def forward(self, inputs):
        # Скалярное произведение + смещение
        z = np.sum(inputs * self.weights) + self.bias
        # Функция активации (сигмоида)
        return 1.0 / (1.0 + np.exp(-z))
```

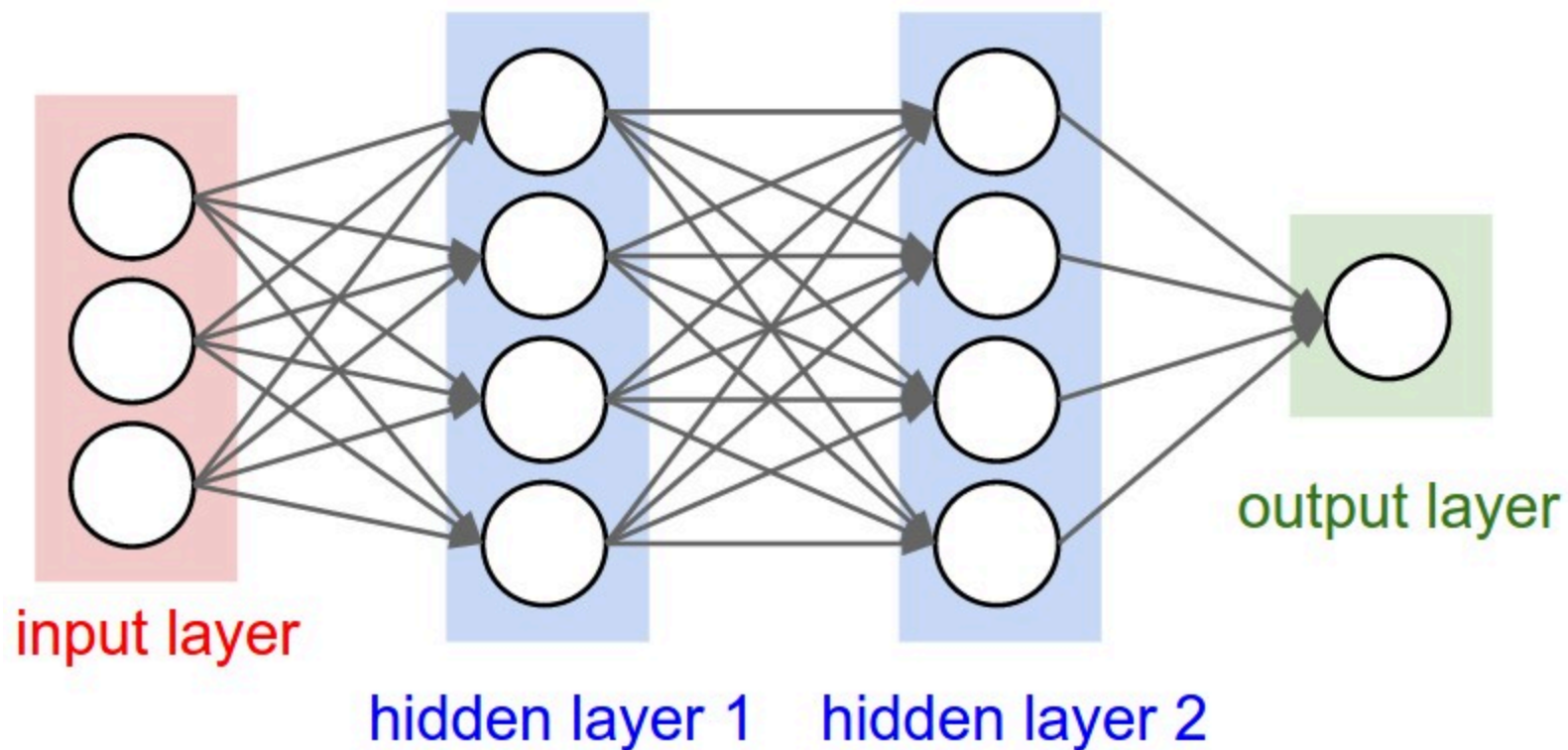
Архитектура нейронных сетей



Архитектура нейронных сетей

Один слой - вектор нейронов

W - матрица слоя, каждая строка матрицы - веса одного нейрона



Архитектура нейронных сетей

Полносвязные слои (Fully-Connected):

```
Вход [3×1] → [W1:4×3] + b1 → ReLU → [4×1]  
           → [W2:4×4] + b2 → ReLU → [4×1]  
           → [W3:1×4] + b3 → Выход [1×1]
```

Пример forward pass:

```
f = lambda x: np.maximum(0, x) # ReLU  
  
h1 = f(W1 @ x + b1) # Скрытый слой 1  
h2 = f(W2 @ h1 + b2) # Скрытый слой 2  
out = W3 @ h2 + b3   # Выход (без активации!)
```


Архитектура нейронных сетей

Представительная способность:

Теорема универсальной аппроксимации:

Сеть с ≥ 1 скрытым слоем может аппроксимировать любую непрерывную функцию с любой точностью.

Но на практике глубокие сети работают лучше благодаря иерархическому представлению признаков!

Прямое (forward) распространение: матричные операции

Почему слои? Эффективность вычислений

Один слой = Матричное умножение + Смещение + Активация

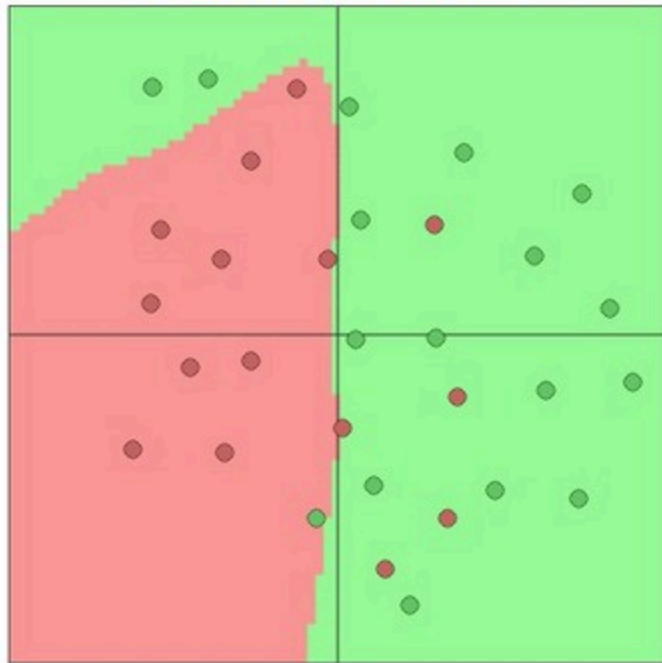


Пакетная обработка (batch):

```
# x: [D×N] — N примеров в столбцах  
h1 = f(W1 @ X + b1) # Все примеры параллельно!
```

Архитектура нейронных сетей

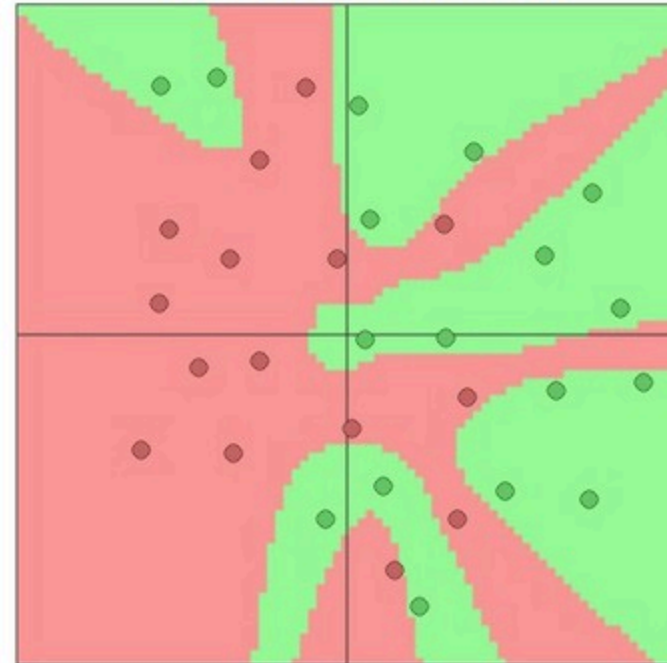
3 hidden neurons



6 hidden neurons

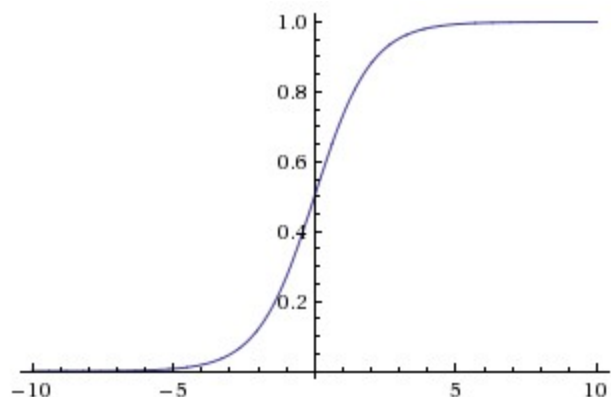


20 hidden neurons



Функции активации: сравнение

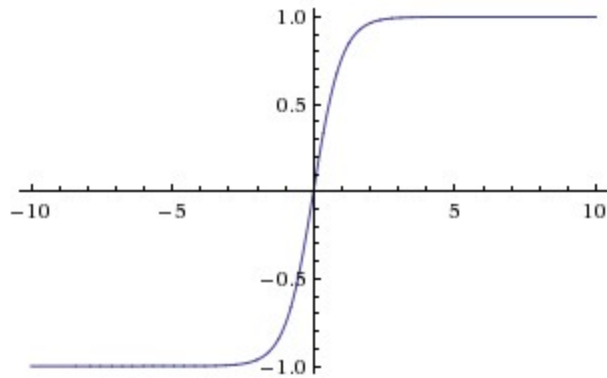
◆ Sigmoid: $\sigma(x) = 1/(1+e^{-x})$



- Диапазон: $[0, 1]$
- Насыщение $\rightarrow$ градиенты ≈ 0
- Выход не центрирован вокруг 0
- Редко используется в скрытых слоях

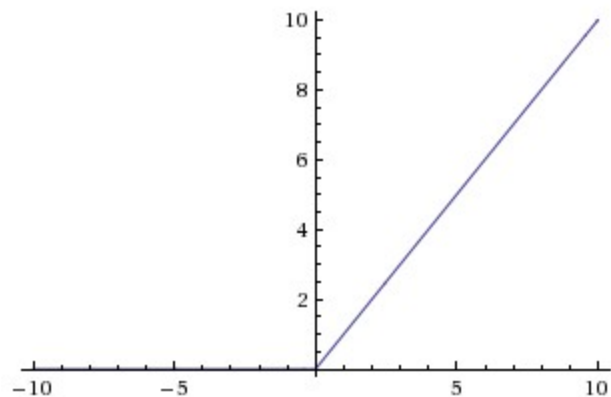
Функции активации: сравнение

◆ Tanh: $\tanh(x)$



- Диапазон: $[-1, 1]$ ✓
- ✓ Выход центрирован вокруг 0
- Всё ещё страдает от насыщения
- Лучше сигмoиды, но уступает ReLU

Функции активации: сравнение



◆ ReLU: $f(x) = \max(0, x)$

- ✓ Быстрая вычислительно
- ✓ Ускоряет сходимость в 6× (по сравнению с tanh)
- "Мёртвые нейроны" при высоком LR
- Стандарт де-факто для скрытых слоёв

Датасет:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad x_i \in \mathbb{R}^D, \quad y_i \in \{1, \dots, K\}$$

Архитектура сети (2 слоя для наглядности):

$$z_1 = W_1 x + b_1 \quad [\text{линейный слой 1}]$$

$$h = \text{ReLU}(z_1) \quad [\text{активация}]$$

$$z_2 = W_2 h + b_2 \quad [\text{линейный слой 2}]$$

$$s = z_2 \quad [\text{логиты, оценки классов}]$$

Цель:

$$\min_{W_1, b_1, W_2, b_2} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell(s^{(i)}, y_i) + \lambda R(W)$$

Chain rule

Для скалярной функции композиции:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Для векторных функций (Якобианы):

$$\frac{\partial \mathbf{u}}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{v}} \cdot \frac{\partial \mathbf{v}}{\partial \mathbf{x}}$$

Ключевая идея backprop: **Градиент потери по параметру = произведение локальных градиентов по пути от выхода к параметру**

$$\begin{array}{ccccccc} L & \leftarrow & s & \leftarrow & h & \leftarrow & z_1 & \leftarrow & W_1 \\ & & \uparrow & & \uparrow & & \uparrow & & \uparrow \\ & & \partial L / \partial s & & \partial s / \partial h & & \partial h / \partial z_1 & & \partial z_1 / \partial W_1 \end{array}$$

Прямой (forward) проход: детализация

Для одного примера (x, y) :

```
# Layer 1: Linear + ReLU
z1 = w1 @ x + b1                                # [H×1]
h = np.maximum(0, z1)                            # [H×1], ReLU

# Layer 2: Linear → logits
s = w2 @ h + b2                                # [K×1]

# Softmax для вероятностей
exp_s = np.exp(s - np.max(s))                  # численная стабильность
probs = exp_s / np.sum(exp_s)                  # [K×1],  $\sum probs = 1$ 
```

Матричная форма для батча $X \in \mathbb{R}^{D \times N}$:

$$Z_1 = W_1 X + b_1 \mathbf{1}^T$$

$$H = \max(0, Z_1)$$

$$S = W_2 H + b_2 \mathbf{1}^T$$

$$P = \text{softmax}(S)$$

Функция потерь: Negative Log-Likelihood

Определение для одного примера:

$$\ell(s, y) = -\log P(y|x) = -\log \left(\frac{e^{s_y}}{\sum_{k=1}^K e^{s_k}} \right)$$

Упрощение (log-sum-exp trick):

$$\ell(s, y) = -s_y + \log \left(\sum_{k=1}^K e^{s_k} \right)$$

Функция потерь: Negative Log-Likelihood

Для всего датасета с L2-регуляризацией:

$$\mathcal{L} = \underbrace{-\frac{1}{N} \sum_{i=1}^N s_{y_i}^{(i)}}_{\text{кросс-энтропия}} + \underbrace{\frac{1}{N} \sum_{i=1}^N \log \left(\sum_k e^{s_k^{(i)}} \right)}_{\text{нормировка}} + \underbrace{\frac{\lambda}{2} (\|W_1\|_F^2 + \|W_2\|_F^2)}_{\text{регуляризация}}$$

Обратный проход: градиент по логитам s

Вычисляем $\frac{\partial \ell}{\partial s_j}$ для всех $j \in \{1, \dots, K\}$:

$$\frac{\partial \ell}{\partial s_j} = \frac{\partial}{\partial s_j} \left[-s_y + \log \left(\sum_k e^{s_k} \right) \right]$$

Случай 1: $j = y$ (правильный класс):

$$\frac{\partial \ell}{\partial s_y} = -1 + \frac{e^{s_y}}{\sum_k e^{s_k}} = \boxed{p_y - 1}$$

Случай 2: $j \neq y$ (неправильные классы):

$$\frac{\partial \ell}{\partial s_j} = 0 + \frac{e^{s_j}}{\sum_k e^{s_k}} = \boxed{p_j}$$

Обратный проход: градиент по логитам s

Итог в векторной форме:

$$\frac{\partial \ell}{\partial s} = p - \mathbf{1}_y$$

где $\mathbf{1}_y$ — one-hot вектор с единицей на позиции y .

Обратный проход: градиенты по W_2, b_2

Используем цепное правило:

$$\frac{\partial \ell}{\partial W_2} = \frac{\partial \ell}{\partial s} \cdot \frac{\partial s}{\partial W_2}, \quad s = W_2 h + b_2$$

Поскольку $s = W_2 h + b_2$ — линейная функция:

$$\frac{\partial s}{\partial W_2} = h^T \quad (\text{по правилам дифференцирования матриц})$$

Градиенты для одного примера:

$\frac{\partial \ell}{\partial W_2} = (p - \mathbf{1}_y) \cdot h^T$	$[K \times H]$
$\frac{\partial \ell}{\partial b_2} = p - \mathbf{1}_y$	$[K \times 1]$

Обратный проход: градиент через ReLU

Прямой проход: $h = \max(0, z_1)$

Производная ReLU (поэлементно):

$$\frac{\partial h_j}{\partial z_{1,j}} = \begin{cases} 1 & \text{if } z_{1,j} > 0 \\ 0 & \text{if } z_{1,j} \leq 0 \end{cases} = \mathbf{1}_{z_1 > 0}$$

Градиент по z_1 (цепное правило):

$$\boxed{\frac{\partial \ell}{\partial z_1} = \frac{\partial \ell}{\partial h} \odot \mathbf{1}_{z_1 > 0}}$$

где $\odot$ — поэлементное произведение (Hadamard product).

Вычисляем $\frac{\partial \ell}{\partial h}$ из предыдущего слоя:

$$\frac{\partial \ell}{\partial h} = W_2^T \cdot \frac{\partial \ell}{\partial s} = W_2^T (p - \mathbf{1}_y)$$

Аналогично второму слою:

$$\frac{\partial \ell}{\partial W_1} = \frac{\partial \ell}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1}, \quad z_1 = W_1 x + b_1$$

Итоговые формулы для одного примера:

$\delta_1 = [W_2^T (p - \mathbf{1}_y)] \odot \mathbf{1}_{z_1 > 0}$	$[H \times 1]$
$\frac{\partial \ell}{\partial W_1} = \delta_1 \cdot x^T$	$[H \times D]$
$\frac{\partial \ell}{\partial b_1} = \delta_1$	$[H \times 1]$

Пример: расчёт на мини-батче

Данные:

$D=2$ (признаки), $H=3$ (скрытые нейроны), $K=2$ (класса), $N=2$ (батч)
 $X = \begin{bmatrix} [1, 2], \\ [0, 1], \\ [1, 0] \end{bmatrix}$ $Y = [0, 1]$ (метки)
($D \times N = 3 \times 2$)

Прямой проход (упрощённо):

$W1 = \begin{bmatrix} [0.1, 0.2, 0.3], \\ [0.4, 0.5, 0.6] \end{bmatrix}$ $b1 = [0, 0, 0]$
(2×3)

$Z1 = W1 \cdot X = \begin{bmatrix} [0.5, 0.9], \\ [1.0, 1.8], \\ [1.5, 2.7] \end{bmatrix}$ $H = \text{ReLU}(Z1) = Z1$ (все >0)

Выходной слой:

```
W2 = [[0.1, 0.2, 0.3],    b2 = [0, 0]
      [0.4, 0.5, 0.6]]    (2x3)

S = W2 · H = [[1.4, 2.4],    Probs = softmax(S)
              [2.3, 3.9]]    ≈ [[0.29, 0.21],
                               [0.71, 0.79]]
```

Градиент по логитам:

```
ds = Probs - one_hot(Y) = [[0.29-1, 0.21-0],    = [[-0.71, 0.21],
                          [0.71-0, 0.79-1]]    [ 0.71, -0.21]]
```

Продолжение вычислений по формулам выше...

Итоги: ключевые формулы backprop

Для выхода (softmax + NLL):

$$\frac{\partial \mathcal{L}}{\partial s} = \frac{1}{N} (P - Y)$$

Для линейного слоя $s = Wx + b$:

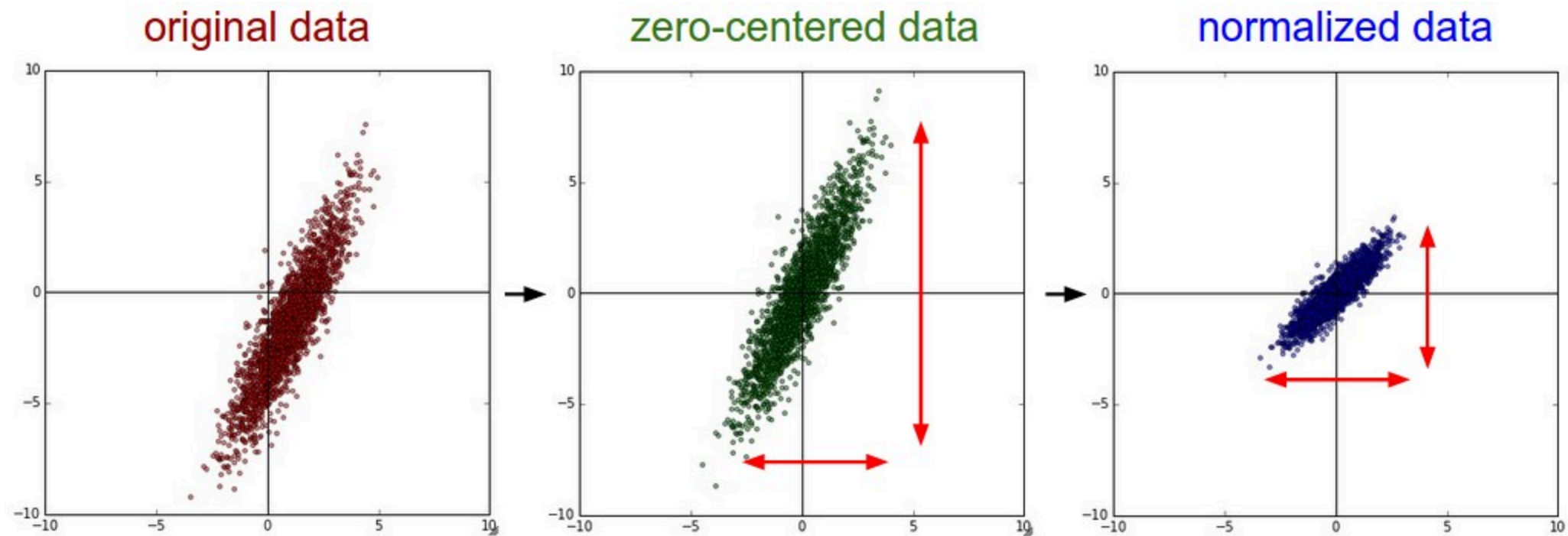
$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W} &= \frac{\partial \mathcal{L}}{\partial s} \cdot x^T + \lambda W \\ \frac{\partial \mathcal{L}}{\partial b} &= \sum \frac{\partial \mathcal{L}}{\partial s} \\ \frac{\partial \mathcal{L}}{\partial x} &= W^T \cdot \frac{\partial \mathcal{L}}{\partial s}\end{aligned}$$

Итоги: ключевые формулы backprop

Для ReLU $h = \max(0, z)$:

$$\boxed{\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial h} \odot \mathbf{1}_{z>0}}$$

Предобработка данных



Предобработка данных

◆ Центрирование (Mean Subtraction):

```
X -= np.mean(X, axis=0) # По каждому признаку  
# Для изображений иногда: X -= np.mean(X)
```

Геометрический смысл: центр облака данных в начале координат

◆ Нормализация (Normalization):

```
# Вариант 1: по стандартному отклонению  
X /= np.std(X, axis=0)  
  
# Вариант 2: масштабирование в [-1, 1]  
X = 2 * (X - X.min()) / (X.max() - X.min()) - 1
```

Инициализация весов

✗ Инициализация нулями:

Все нейроны вычисляют одинаковый выход →
одинаковые градиенты → одинаковые обновления →
симметрия не нарушается → сеть не обучается!

✓ Малые случайные числа:

```
W = 0.01 * np.random.randn(n_in, n_out)
```


Инициализация весов

Калибровка дисперсии:

Метод	Формула	Когда использовать
Xavier/Glorot	$W \sim N(0, 2/(n_{in}+n_{out}))$	Tanh, Sigmoid
He/Kaiming ★	$W \sim N(0, 2/n_{in})$	ReLU (рекомендуется)

Смещения (biases):

```
b = np.zeros(n_units) # Обычно достаточно  
# Иногда для ReLU: b = 0.01 (чтобы нейроны "включились")
```

Регуляризация: борьба с переобучением

◆ L2-регуляризация (Weight Decay):

$$L = L_{\text{data}} + \frac{1}{2}\lambda \cdot \sum w^2$$

Градиент: $\partial L / \partial w = \partial L_{\text{data}} / \partial w + \lambda \cdot w$

Обновление: $W += -\text{lr} \cdot (\text{grad} + \lambda \cdot W)$ ← "затухание" весов

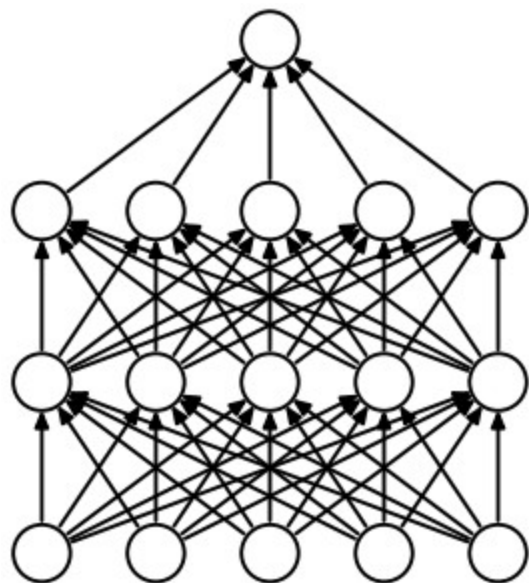
Предпочитает распределённые веса, а не "пиковые"

◆ L1-регуляризация:

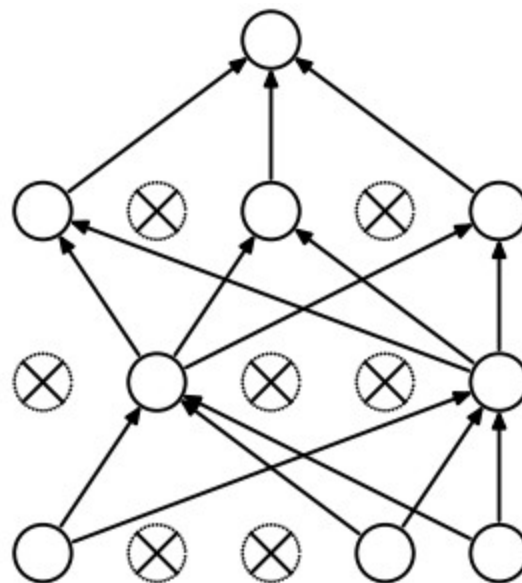
$$L = L_{\text{data}} + \lambda \cdot \sum |w|$$

Приводит к разреженным весам → отбор признаков

Dropout: борьба с переобучением



(a) Standard Neural Net



(b) After applying dropout.

Обучение:

```
mask = np.random.rand(*H.shape) < p # p = 0.5
```

```
H *= mask # "Выключаем" нейроны
```

```
H /= p # Инвертный dropout (масштабирование)
```

Инференс: ничего не делаем (масштабирование уже учтено)

Оптимизация: методы обновления весов

Vanilla SGD:

```
W += -learning_rate * grad
```

Просто, но может "застревать" в локальных минимумах

Momentum:

```
v = mu * v - lr * grad # mu ≈ 0.9  
W += v
```

Добавляет "инерцию" → быстрее проходит плато

Адаптивные методы оптимизации

◆ Adagrad:

```
cache += grad**2  
W += -lr * grad / (np.sqrt(cache) + eps)
```

Адаптирует LR для каждого параметра, но LR монотонно убывает

◆ RMSProp:

```
cache = beta * cache + (1-beta) * grad**2  
W += -lr * grad / (np.sqrt(cache) + eps)
```

Использует скользящее среднее → решает проблему Adagrad

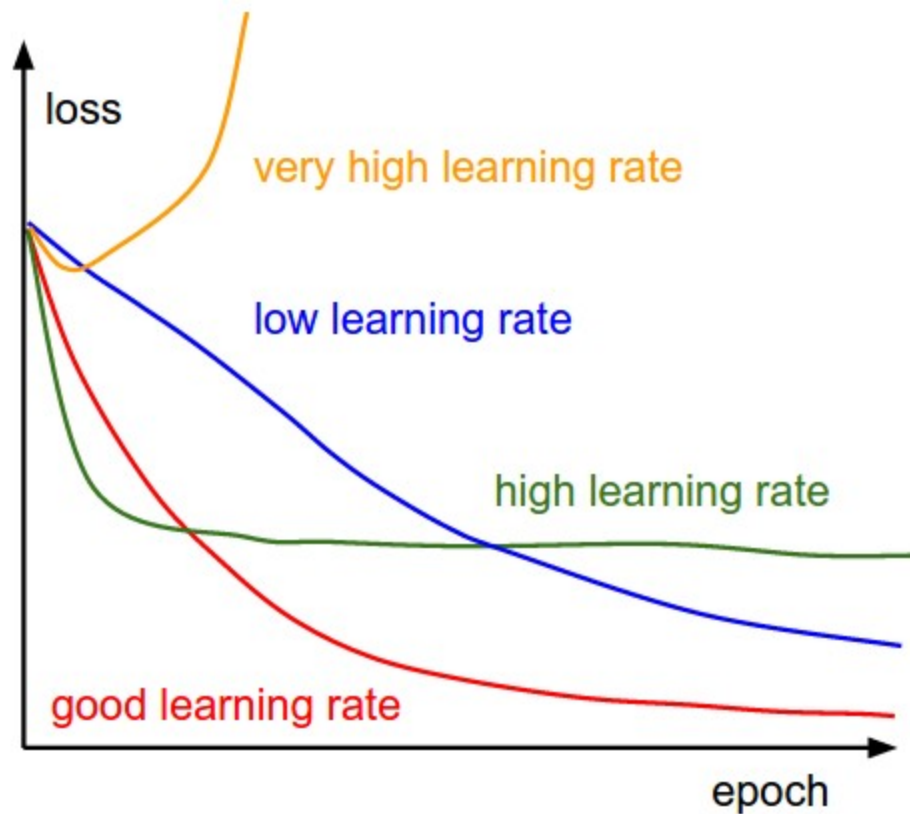
Адаптивные методы оптимизации

◆ Adam (рекомендуется по умолчанию):

```
m = beta1*m + (1-beta1)*grad      # Первый момент (mean)
v = beta2*v + (1-beta2)*grad**2    # Второй момент (variance)
W += -lr * m / (np.sqrt(v) + eps)
```

Рекомендуемые гиперпараметры: $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=10^{-8}$

Мониторинг обучения



Что делать при переобучении?

1. Увеличить регуляризацию (L2, Dropout)
2. Собрать больше данных
3. Уменьшить сложность модели (если данных мало)

Практические рекомендации

Чеклист для старта:

- [] Предобработка: центрирование + нормализация
- [] Инициализация: Не для ReLU, Xavier для tanh
- [] Активации: ReLU в скрытых слоях, линейная на выходе
- [] Оптимизатор: Adam с $lr=3e-4$ (стартовая точка)
- [] Регуляризация: Dropout $p=0.5$ + L2 $\lambda=1e-4$
- [] Мониторинг: loss, train/val accuracy, градиенты

Практические рекомендации

Градиентный чек (для отладки):

```
# Используйте центральную разность!  
grad_num = (f(x+h) - f(x-h)) / (2*h)  
  
# Сравнивайте через относительную ошибку:  
error = |grad_analytic - grad_num| / max(|grad_analytic|, |grad_num|)  
# ✓ Хорошо: error < 1e-7  
# Подозрительно: 1e-7 < error < 1e-4  
# Ошибка: error > 1e-4
```

Итоги

Ключевые выводы:

1. Нейрон = линейное преобразование + нелинейность
2. Глубокие сети = композиция простых преобразований
3. Нелинейности (ReLU) критичны для выразительной силы
4. Правильная инициализация и регуляризация — залог успеха
5. Мониторинг метрик важнее "угадывания" гиперпараметров

Дополнительные ресурсы:

- [CS231n Notes](#)
- [Deep Learning Book \(Goodfellow et al.\)](#)
- [Distill.pub](#) — визуальные объяснения